Frozen-Weight Policy Transfer with Adaptable Mathematical Interfaces: A Benchmark Study of the Neuro-Mathematical Transfer Engine on Differential-Drive Navigation

Given Name Surname, Given Name Surname
dept. name of organization (of Affiliation)
name of organization (of Affiliation)
City, Country
email address or ORCID

ABSTRACT— Transfer learning can already carry robot skills across domains; what is less understood is how far a *frozen* learned controller can be carried into new worlds when only lightweight mathematical interfaces are allowed to adapt. We study this question with the Neuro-Mathematical Transfer Engine (NMTE) program, whose working hypothesis is a division of labour: a learned policy supplies *competence*, while mathematical interfaces (geodesic waypoint guidance, disturbance observation, barrier-based safety filtering) supply the *guarantees and task grounding* that do not transfer by themselves. We instantiate the program on a differential-drive navigation benchmark in Isaac Lab / Isaac Sim with a ROS 2 deployment rung: one policy, trained once in 8 m rooms with scattered walls, is redeployed with frozen weights across unseen geometry, unseen task structure, degraded dynamics and a second embodiment. We report positives and negatives with equal care. The frozen policy transfers zero-shot along a measured gradient (a difficulty ladder, 4/4 successes under spawn jitter), and an adaptation-budget curve places its zero-shot performance at the level of a from-scratch behaviour-cloned policy trained on roughly 238 k target-world demonstrations. A deploy-time safety shield turns contact-free operation from an emergent hope into a structural property: across five presentation maps and two independent serial-goal runs, an exact-geometry audit finds zero hull-penetration frames, where the unshielded policy produced grazing contacts, one sustained wedge failure and post-success grinding. A systematic three-arm campaign on 56 procedural maps (708 episodes) sharpens the division of labour into the paper's central picture: an in-house A*+DWA planner wrapped in the same shield scores 0.979 goal success with *zero* collisions in 236 episodes — better success than the unprotected planner (0.941), which contacts walls in 53.4 % of episodes — while the trained policy's competence proves family-structured (0.94 on open families, 0.00 on narrow corridors) and its 84 failures are 83 timeouts plus one marginal scrape: the guarantee comes from the interface and is arm-agnostic. A held-out endgame — a single maze drawn by a user, unseen in training and the campaign — sharpens the point: the same shield-wrapped planner is 10/10 successful with 0/10 collisions (extending the guarantee to 0/246 episodes), while the frozen learned policy reaches 0/10 and its unprotected planner twin collides in 9/10. We state limitations plainly: simulation-only validation, a fixed-seed campaign protocol (spawn-yaw repeats per map), an observation-relative safety guarantee, and one framework arm (cross-embodiment optimal transport) left unexercised. The contribution is not a new planner and not a claim that transfer learning is unnecessary; it is a measured case study of *what a frozen learned core plus adaptable mathematical interfaces can and cannot do*.

Keywords—Transfer learning, sim-to-real, frozen-weight deployment, control barrier functions, disturbance observers, Koopman operators, differential-drive navigation, safety filters.

I. INTRODUCTION

Deep reinforcement learning (DRL) produces capable robot controllers inside physics simulators, but deploying them outside the training world degrades performance — the reality gap. The dominant remedies, domain randomization and target-domain fine-tuning, spend data and compute at every new deployment. Transfer learning in the classical sense (adapting weights with target data) already works for many of these problems, and we do not dispute it. The question this paper asks is narrower and, we argue, complementary: *how much of the cross-world burden can be carried without touching the weights at all?* If a policy trained once can be redeployed by adapting only small, interpretable, certifiable interfaces, then each new world costs interface work rather than a training campaign — and the safety-critical part of the stack becomes mathematical rather than statistical.

This is the operating hypothesis of the Neuro-Mathematical Transfer Engine (NMTE) program, first formulated as a mathematical framework (Koopman operator core, Gromov–Wasserstein cross-embodiment alignment, disturbance-observer-based control barrier functions) and instantiated here on a differential-drive navigation benchmark. The instantiation is deliberately modest: a 2-wheel robot, LiDAR and odometry, planar maps. The modesty is the point — it lets us measure the transfer-first paradigm end to end, including its failures, on hardware-class compute.

The paper makes five concrete contributions, each backed by kept artifacts (traces, logs, scripts):

1. **A frozen-weight transfer benchmark.** One checkpoint, trained solely in a source world of scattered-wall rooms, is evaluated across four shift axes (geometry, task structure, dynamics, embodiment) with zero gradient steps in any target world (Sec. VI).
2. **A measured transfer gradient and boundary.** A difficulty ladder with identical spawn, goal and path length shows that transfer degrades with *behavioural* distance from the source distribution, not with visual complexity; seeded repeats and a waypoint-blinding ablation separate robust transfer from lucky seeds and locate which interface carries it (Sec. VI-B).
3. **A quantified adaptation budget.** A retrain-from-scratch curve on the target world crosses the frozen engine's zero-shot performance at ≈238 k expert demonstrations, giving a number — with pre-disclosed caveats — for what transfer-first saves (Sec. VI-B).
4. **A deploy-time safety shield with a structural no-contact**

property. A two-tier velocity-barrier filter between policy and chassis makes "move, stop, do whatever — but never collide" true by construction; an exact-geometry audit verifies zero contact frames across the whole presentation set, a three-arm campaign on 56 procedural maps (708 episodes) shows the guarantee is arm-agnostic — zero collisions for a shield-wrapped classical planner that collides in 53.4 % of episodes unwrapped, and even gains success (0.941 → 0.979) — and the shield's own seven-version iteration history (including one livelock, one absorbing freeze and their fixes) is kept as evidence (Sec. IV-D, VI-E, VI-G).

5. **An honest accounting.** Every negative result — failed training runs, a deadlocked shield version, a no-op safety filter variant, an unhelpful Koopman lift, an unreachable map class — is reported alongside the positives, because the boundary of a transfer method is part of its result (Sec. VII-B).

The remainder of the paper is organised as follows. Sec. II positions the work against transfer learning, domain randomization and safety-filtered learning. Sec. III describes how the framework idea evolved from the first draft to the benchmark. Sec. IV gives the system and methods, including the shield formalisation. Sec. V defines the experimental setup and the audit methodology. Sec. VI reports results axis by axis. Sec. VII discusses what is proven, what is not, and how the framework serves the broader "learning can be transferred" goal. Sec. VIII concludes.

II. RELATED WORK AND POSITIONING

A. Transfer learning and domain randomization

Transfer learning (TL) — reusing source-world knowledge with target-world adaptation — is an established and effective route across the reality gap: recent surveys treat it as the most promising direction for cross-embodiment robotics [1], and sim-to-real surveys cover the same ground for manipulation and locomotion [16]. Domain randomization (DR) trades target data for widened source distributions; it helps but leaves a residual distributional shift at deployment [2]. Cross-embodiment skill transfer via latent-space alignment is actively developed [3], [18], [19]. Our work is not a competitor to any of these: it fixes the weights entirely and asks what interfaces alone can carry. Where TL spends target data, we spend target *structure* (a map file, an observer, a barrier); the adaptation-budget curve of Sec. VI-B is precisely the exchange rate between the two paradigms on one axis.

B. Safety-filtered learning

Control barrier functions (CBFs) provide forward-invariance certificates for safe sets [9]; combining them with learned policies through a minimally invasive quadratic program is standard practice, and disturbance-observer-based CBFs extend the certificate to unmodelled dynamics [4]. Minkowski-type barriers handle polytopic footprints and obstacle sets [5]. At the planning level, the dynamic window approach (DWA) filters velocities by forward-simulated stoppability [8]. Our deploy-time shield is deliberately of this family — a DWA-style candidate filter with a hull-aware, persistence-backed obstacle model and a two-tier barrier — because the research question is not a new filter but *what a classical guarantee arm buys when wrapped around a frozen learned policy at deployment time*. Earlier training-world A/B tests of a minimal cone barrier (CBF-lite) are reported as concept checks with their own negative (Sec. VI-D).

C. Operator-theoretic dynamics models

Koopman operator theory lifts nonlinear dynamics into linearly evolving observables, enabling cheap linear prediction and control [6], [7], [14]. We use a frozen extended-dynamic-mode-analysis (EDMD) lift as a concept check on the dynamics axis: which channels of a frozen source-world model survive a dynamics shift, and what a small refit recovers (Sec. VI-C). The honest finding is that at this scale the lift adds little over an affine model; what survives is the *freeze-structure / adapt-interface* pattern, not the lifting mechanism.

D. Research gaps addressed

The first draft of this program identified five gaps: zero-shot transfer in contact-rich settings, task-invariant features under degraded terrain, parameter-efficient cross-embodiment alignment, certifiable safety during adaptation, and edge-compute constraints [1], [4], [16], [17]. This paper addresses the safety-certification gap with a deployed, audited mechanism; the geometry/task gaps with measured zero-shot transfer and ablations; the dynamics gap with a disturbance observer and an operator-theoretic check; and the edge gap with a cost measurement of the shield (≈14 ms per intervention on CPU at 20 Hz). The cross-embodiment alignment gap is explicitly left open (Sec. VII-B).

III. FROM FRAMEWORK DRAFT TO BENCHMARK: HOW THE IDEA EVOLVED

The first draft proposed NMTE as a mathematical architecture: a Koopman core for task-invariant dynamics, Gromov–Wasserstein (GW) optimal transport for cross-embodiment latent alignment, and disturbance-observer-based CBFs for certifiable safety, wrapped around a neural feature extractor. Building a first instantiation changed the framing in three ways, and the second draft should reflect the change rather than hide it.

First, the invariant is the division of labour, not the mechanisms. The named components (Koopman core, DOB, CBF, GW alignment) are concepts; in the benchmark each was instantiated in the simplest form that could be validated or falsified (a scalar plant-gain observer, a velocity-barrier filter, a quadratic EDMD lift; GW not at all). What survived every substitution is the principle: *the learned policy provides competence; the mathematical interface provides guarantees and task grounding; the weights never move.* Fig. 1 states the principle as the system architecture.

Second, competence itself needed an interface to exist. Pure PPO never bootstrapped blocked-goal reaching in the source world (Sec. VI-A); the policy became competent only after a geodesic-waypoint observation channel was introduced. Transfer-first therefore starts earlier than deployment: the *training* interface (mathematical guidance in observation space) is what makes the frozen core worth transferring. The later waypoint-blinding ablation closes the loop: removing that channel at deployment collapses transfer (Sec. VI-B).

Third, the guarantee arm had to be earned at deployment,

not assumed. The first presentation runs (iteration 1) deployed the competence arm alone and reached every goal — while touching walls in almost every video. Auditing that flaw (Sec. VI-E) quantified exactly the gap the framework draft had predicted: learned policies cannot certify constraint satisfaction. Iteration 2 added the shield and re-ran the same maps with the same weights; the contact went to zero. The iteration-1 videos are kept as the diagnostic, the iteration-2 videos as the fix.

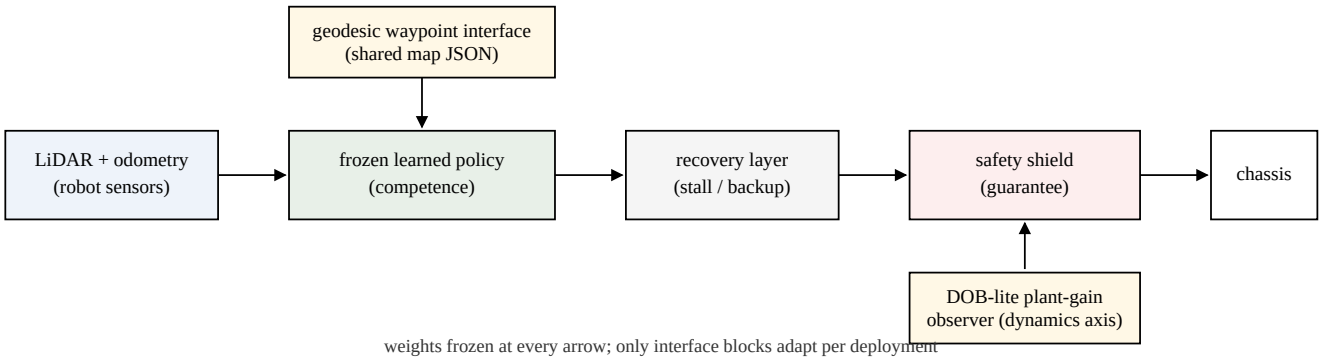


Fig. 1. The division of labour that survived every mechanism substitution: a frozen learned policy emits velocity commands; a recovery layer handles stalls; a deploy-time safety shield has final authority over the chassis. Mathematical interfaces (waypoint guidance from the shared map file, plant-gain disturbance observation) adapt per deployment without touching the weights. *Placement: full-width (figure*), page 2. Diagram drawn inline as SVG; regenerate as vector PDF for submission.*

IV. SYSTEM AND METHODS

A. Source world and the frozen policy

The source task is point-goal navigation for a differential-drive robot (openmover: 2 × 600-bin LiDAR at ±90° front/rear, wheel radius 0.0504 m, separation 0.324 m) in 8 m rooms with 4–12 scattered walls, sampled so that every goal chord crosses at least one wall (blocked-goal sampling). Training followed 17 logged runs (≈50 M environment steps) in Isaac Lab [11], [12] with PPO [13]; the chronology and its negatives are in Sec. VI-A. The deployed policy is a behaviour clone of a validated geodesic expert under a corrected observation, with a contact-force channel: an MLP mapping a 1208-dimension observation (front/rear scans, goal bearing/distance, proprioception, geodesic waypoint and geodesic distance, contact channel) to two wheel commands at 20 Hz. The checkpoint (`bc_init_v3/model_0.pt`) is frozen for every result in this paper.

B. Deployment rung and the shared world model

Deployment runs through plain ROS 2 topics [20] (Fig. 2): an Isaac Sim bridge publishes scan and odometry and consumes `/cmd_vel`; a policy node closes the loop. A single map JSON file is the shared world model: the bridge bakes its wall boxes into simulation colliders, and the policy node builds a geodesic field (GeoField) from the same boxes, exposing a waypoint and geodesic distance toward the goal as observation channels. Maps are authored by drawing a PNG and converting it through a BFS-validated pipeline (Fig. 3), so "a new world" costs one drawing, not one training run. Serial multi-goal tasks are composed at the interface: an ordered goal list advances on arrival and rebuilds the GeoField per leg, with zero weight changes.

C. Interface mechanisms

Geodesic waypoint interface (task/geometry axis). The GeoField waypoint is the task carrier across worlds; Sec. VI-B's blinding ablation shows it is load-bearing.

DOB-lite (dynamics axis). A scalar plant-gain disturbance observer [15] sits between policy and actuators: it compares commanded and realised wheel joint speed, identifies the thrust-gain loss online, and rescales the wheel command by 1/g. Its fixed point is provably independent of its own correction, and g = 1 is a bit-exact no-op.

Safety filter (safety axis). Two generations: CBF-lite, a minimal directional cone barrier used for training-world A/B concept checks (Sec. VI-D), and the deploy-time shield of Sec. IV-D, which supersedes it for presentation runs.

Koopman pipeline (dynamics model, offline). A 12-dimensional quadratic EDMD lift of the state [body twist, realised wheel speeds] with ridge regression, fitted on source-world data and evaluated one-step on degraded-world data (Sec. VI-C).

D. The deploy-time safety shield

The shield is a candidate filter over the policy's velocity command, placed after the recovery layer so that backup manoeuvres are filtered too. Let V×W be a discrete command set (8 linear × 7 angular velocities, including reverse). A candidate (v, w) is *admissible* iff a constant-curvature forward simulation of the 17-point hull footprint (0.44 × 0.28 m: corners, edge midpoints, quarters) over a 0.8 s horizon plus a full braking tail (0.6 m/s²) never enters the obstacle grid. The grid is built from *every valid LiDAR hit the sensors have ever seen*, buffered per beam in world frame and never cleared (the maps are static). This last-seen persistence is the key design decision: the scanners have a 0.22 m near clip, so without persistence the shield would go blind exactly when the robot closes on a wall. Tier 1 (comfort) tests against the grid dilated by a 0.10 m margin at 0.08 m resolution. The policy's own command passes iff it is admissible; otherwise the shield substitutes the admissible candidate maximising forward progress with a turn-rate bonus and a minimality penalty toward the policy's intent. The stop command (0, 0) is always admissible, so the shield can never be forced into an unsafe action: the worst case is standing still.

Two-tier barrier (escape hatch). Grazing LiDAR hits scatter a few centimetres off true wall surfaces, and dilation extends the comfort envelope beyond the clearance the hull actually keeps; in

a tight corner pocket this can leave *no* forward candidate admissible in tier 1, producing an in-place rotation livelock (observed, Sec. VI-E). Tier 2 engages only when tier 1 is rotation-locked: candidates are re-checked against the *raw* grid (cells containing actually-sensed wall points) under a 0.20 m/s speed cap. Raw-grid safety — the hull never entering a cell with a sensed wall point — is the hard physical no-contact bar, so the guarantee holds in both tiers; on maps that never rotation-lock, tier 2 is inert by construction.

Park-after-success. The frozen policy has no stop-at-goal behaviour; left running after the final goal it grinds into the goal pocket (a dominant component of iteration-1 contact, Sec. VI-E). After the final goal the node publishes zero twist. This is interface policy, not learned behaviour, and is labelled as such.

Guarantee statement. With the shield active, no command reaches the chassis unless it survives the hull forward simulation against every obstacle ever sensed — at comfort margin (tier 1) or, when tier 1 offers only stop/rotate, against the raw grid at ≤0.20 m/s (tier 2). The guarantee is relative to sensed evidence on a static map (Sec. VII-B). Cost: ≈14 ms per intervention tick on CPU at 20 Hz.

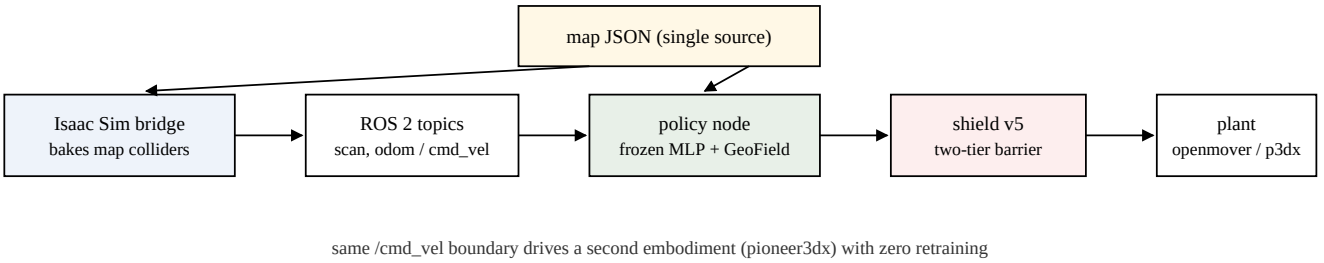


Fig. 2. Deployment chain. One map file serves both the simulator (colliders) and the controller (GeoField); the policy node and shield communicate only through standard ROS 2 messages, which is what makes the second-embodiment substitution (pioneer3dx plant) a configuration change rather than a re-engineering. *Placement: full-width (figure*), page 3.*

V. EXPERIMENTAL SETUP

A. Simulator, maps, protocols

All experiments run in Isaac Lab / Isaac Sim [12] on a single workstation (RTX 3080, 16 GB RAM); the deployment rung is ROS 2 Jazzy. Source evaluations use the task-v2 protocol (256 envs × 2405 steps, seed 42, blocked-goal sampling; headline metric = blocked-goal success). Presentation maps (Fig. 3, Fig. 11) are user-drawn: *demo_open* (scattered walls), *demo_mid* (full-width staggered partitions, ≈2 m gaps), *demo_gauntlet* (same partitions, gaps offset ±0.8–0.9 m off the direct line), *demo_crossings* (rail-yard-style gaps on the spawn–goal line), and *demo_tour* (three ordered goals). The systematic campaign (Sec. VI-G) uses a separate procedural map set: 56 maps from 7 families × 4 difficulty bins × 2 variants, generated by a seeded, BFS-validated generator with per-family motif counts recorded in a manifest; every (map, arm) cell runs in a fresh simulator process at torch seed 42 with 4 spawn-yaw repeats folded into parallel envs (10 on two locked presentation maps), orchestrated end-to-end by one script with per-run wall clocks, return codes and episode-capture checks. Presentation traces are recorded at 15 Hz with per-beam world-frame LiDAR hits; each run is reproducible from the kept launch scripts and logs.

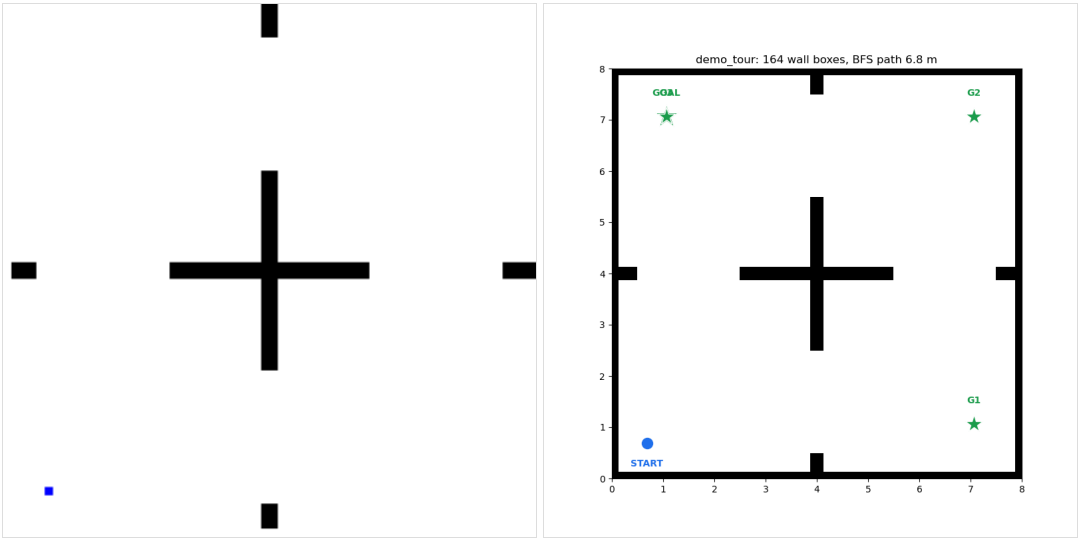


Fig. 3. The map pipeline: a user-drawn PNG (left) is converted to a BFS-validated wall-box JSON and rendered back for inspection (right). The same JSON feeds simulator colliders and the policy's GeoField. *Placement: full-width (figure*), page 4. Source images: openmover_ws/isaac/lab/maps/demo_tour{,.preview}.png.*

B. Contact audit methodology

Claims about contact are audited offline against ground truth, not against the simulator's contact events. For every trace frame,

17 hull sample points are transformed into world frame and tested against the map's wall boxes — the same boxes the bridge bakes into colliders — using exact point-vs-box distance (negative = inside a wall). This *exact-box* mode avoids the over-flagging of raster distance fields used in earlier project documents; all counts in this paper are exact-box and recomputable from the kept traces by the accompanying script. Counts are reported both for the full recorded run and for the approach phase (up to first arrival within the 0.30 m success radius of the final goal), because the unshielded policy's post-success grind is a separate behaviour from approach contact.

C. Reproducibility

Traces, logs, maps, checkpoints and scripts for every number in this paper are kept in the project workspace; the figures are regenerated from artifacts by `draft/make_figures.py`. Negative artifacts (deadlocked shield traces, wedge-failure traces, blinded-ablation traces) are kept alongside positives.

VI. RESULTS

A. What training bought (and what it killed)

The training chronology is part of the result. PPO from scratch never bootstrapped blocked-goal reaching (best 0.095 blocked success); curriculum and clearance shaping did not fix it — exposure was not the constraint. Reward-space geodesic shaping made it worse, and observation-space waypoint injection alone produced a cautious in-place wiggle; both were later found to be confounded by a corrupted waypoint observation (40 % of waypoints pointed through walls), a bug whose discovery invalidated five runs' conclusions and is reported as such. Behaviour cloning from a validated geodesic expert under the corrected observation reached 0.287 blocked success — about 8× the 0.036 drive-straight floor — and DAgger relabelling [10] added nothing once observations were clean: distribution shift is provably not the binding constraint in this pipeline. Two readings matter for the framework: competence required a mathematical interface to exist at all, and most of the compute spent bought falsified hypotheses rather than improvements.

B. Geometry axis: a gradient with a measured boundary

With frozen weights, the policy solves user-drawn maps it never saw: crossings in 33.5 s, ladder easy in 29.3 s, ladder mid in 26.2 s (Fig. 4). The mid rung — full-width partitions, a map class absent from training — is solved *faster* than easy, because the partitions funnel approaches toward the gaps the policy already knows how to thread: transfer is not monotone in visual complexity but in behavioural overlap with the source distribution. The hard rung (gaps offset off the direct line) fails with a corner-wedge signature: the alternating backup recovery cannot change the approach angle enough. Four seeded repeats with spawn jitter (±0.15 m, ±15°) succeed 4/4 (20.9–33.5 s), so the positive side of the gradient is not a lucky seed.

The blinding ablation isolates the carrier: zeroing the geodesic-waypoint and geodesic-distance observation channels on the same map turns 4/4 success into 0/1, the robot pinning 5.74 m short against the first wall. The frozen LiDAR/contact reflexes alone do not compose into gap-threading; the math-guided channel is load-bearing.

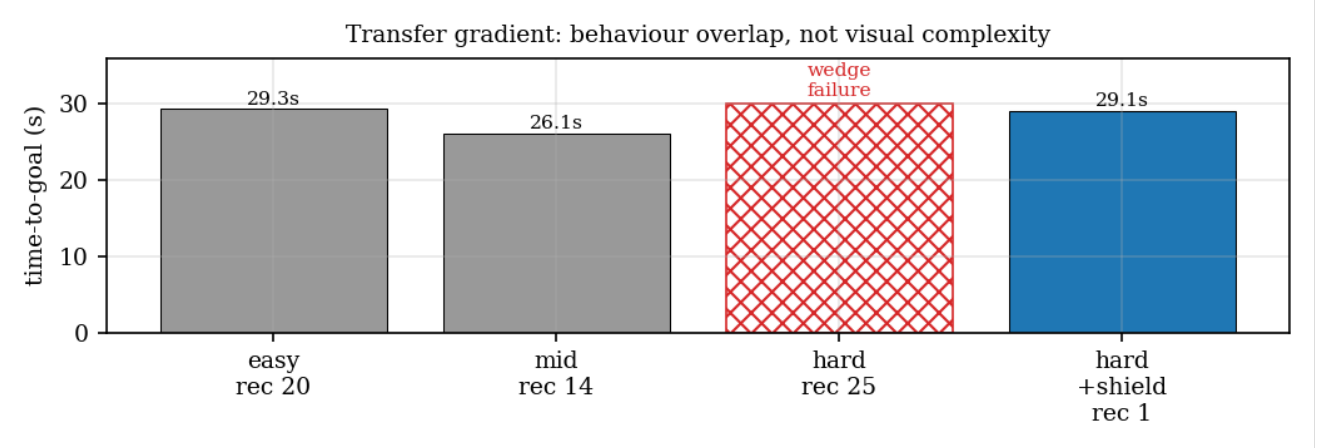


Fig. 4. Difficulty ladder, frozen weights: identical spawn, goal and 13.2 m BFS path; only geometry changes. Easy and mid succeed (mid faster than easy); hard wedges at the second gap corner; adding the deploy-time shield (Sec. VI-E) turns the hard rung into a 29.1 s success with zero contact and one recovery. Recovery counts below the axis. *Placement: full-width (figure*), page 5. Generated by make_figures.py from ledger §13/§15.*

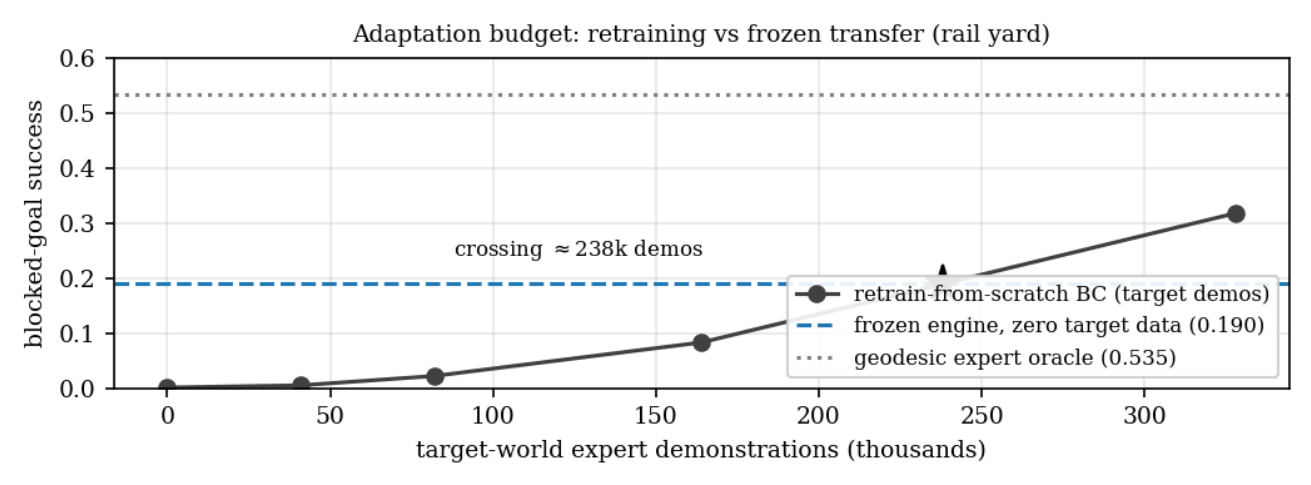


Fig. 5. Adaptation budget on the rail-yard geometry axis: blocked-goal success of from-scratch behaviour cloning against target-world expert-demo budget, versus the frozen engine's zero-shot 0.190 and the geodesic expert oracle 0.535. Linear interpolation places the crossing at ≈238 k demos. Caveats: BC is the most sample-efficient from-scratch baseline available to us (RL would cost more, so 238 k is a lower bound); the curve has not saturated at 328 k; single seed per point. *Placement: full-width (figure*), page 5. Data: ledger §6 table.*

The adaptation-budget curve (Fig. 5) converts the transfer claim into an exchange rate: retraining from scratch on the target world needs on the order of 238 k expert demonstrations to match what the frozen engine delivers with none. With the caveats above, this is the number we offer against "just retrain it".

C. Dynamics axis: the observer adapts, the weights stay frozen

Under degraded actuator gain (thrust ×0.6), the frozen policy's blocked success collapses from 0.287 to 0.028. Adding DOB-lite — no adaptation data, no weight change — restores 0.363, above the nominal frozen baseline, and the identified gain converges to g* = 0.599 against the theoretical 0.6: the interface discovers the degradation factor from runtime data. An oracle arm that knows the factor in advance reaches 0.375, so the online observer lands within 3.2 % of an upper bound it never sees. On combined shift (degraded dynamics + unseen geometry) the observer repairs the dynamics half (0.044 → 0.250) while the geometry half still costs — one interface does not fix both axes, and is not claimed to (Fig. 6, Table I).

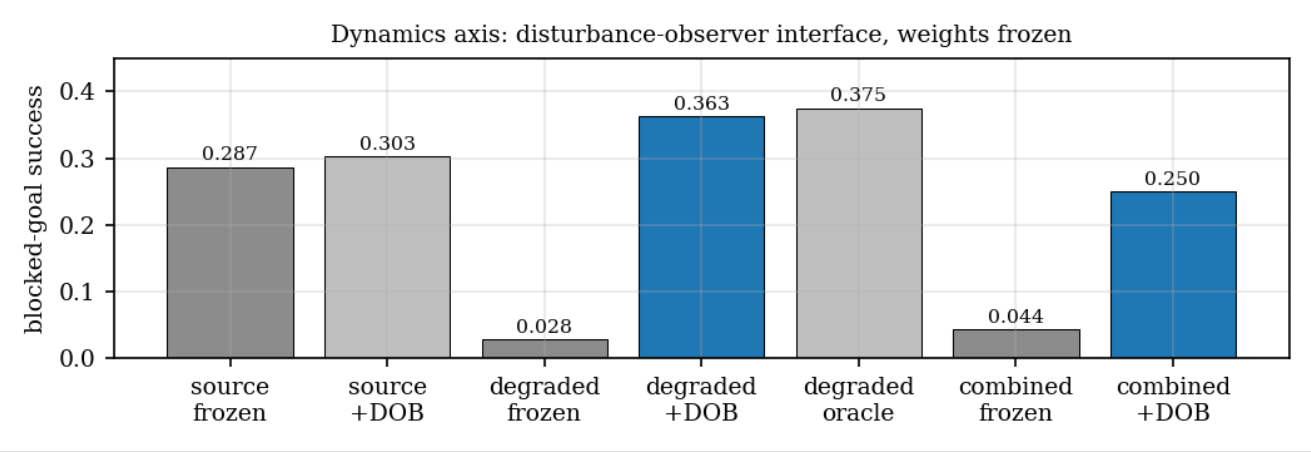


Fig. 6. Dynamics-axis A/B, frozen weights throughout. Degraded: 0.028 → 0.363 with the observer, versus 0.375 for an oracle that knows the degradation in advance. Combined shift: 0.044 → 0.250. *Placement: full-width (figure*), page 6. Data: ledger §7.*

Table I — DOB-lite A/B (blocked-goal success; ledger §7).

deployment	blocked succ.	collision	mean g
source, frozen	0.287	0.642	—
source, +DOB-lite	0.303	0.644	0.964
degraded, frozen	0.028	0.472	—
degraded, +DOB-lite	0.363	0.617	0.599
degraded, oracle boost	0.375	0.629	—
combined, frozen	0.044	0.719	—
combined, +DOB-lite	0.250	0.798	0.602

The Koopman concept check mirrors this on the model side (Fig. 7): a frozen source-world lifted fit keeps the body-twist channels usable on degraded data (R^2 0.76–0.81) but loses the wheel channels (0.88 → 0.57, below the persistence baseline), i.e.

exactly where the degradation acts; a small refit of the lift statistics recovers 0.99. Two honest negatives accompany this: the quadratic lift barely beats a plain affine fit at this scale, and an unstandardised first fit diverged (recorded before the fix). What transfers here is the freeze-structure/adapt-interface pattern, not the lifting mechanism.

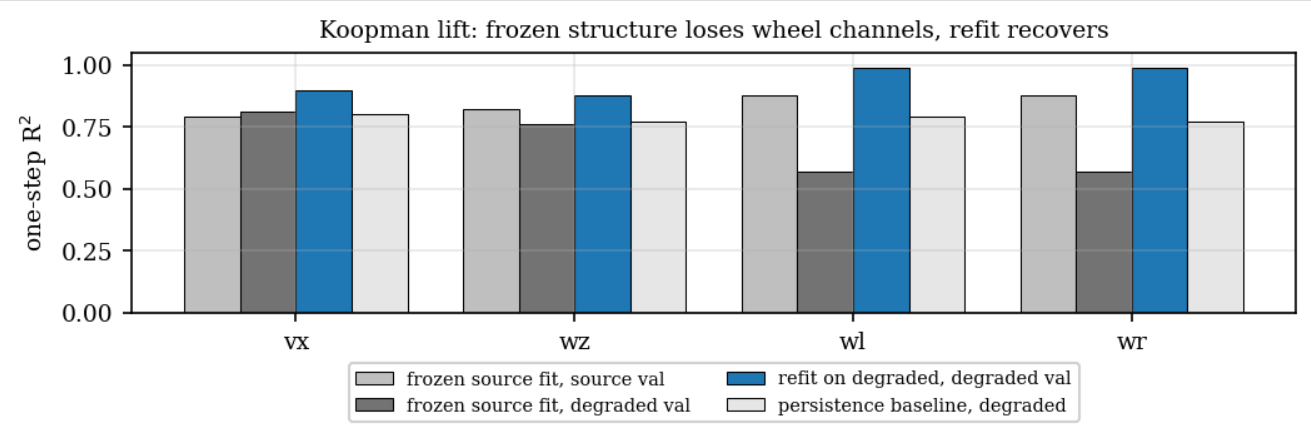


Fig. 7. One-step prediction R^2 per state channel [vx, wz, wl, wr]. The frozen source lift loses the wheel channels on degraded data (0.88 → 0.57, below persistence); refitting the lift statistics on degraded data recovers 0.99. *Placement: full-width (figure*), page 6. Data: ledger §11.*

D. Safety axis, part 1: training-world barrier A/B

A minimal directional-cone barrier (CBF-lite) clamps the forward command to the stoppable speed given LiDAR clearance in the travel direction. On the ring map (Fig. 8) it cuts collisions by 23 % (0.799 → 0.613, three-seed mean) while blocked success slightly improves (0.331 → 0.358): slower approaches near walls reduce bounce-offs that used to end episodes, and the untouched spin component keeps the policy's ability to re-aim. In the source world the filter is nearly free (success 0.287 → 0.274, collisions −11 %). Intervention rate scales with deployment stress: 46–51 % of steps on the out-of-distribution ring versus 14.6 % in-distribution — the filter works hardest where transfer is weakest. The honest negative: on the slalom map, whose failures are side/oblique contacts while threading narrow gaps, a single travel-cone barrier is a no-op (0.2 % interventions, identical 0.000 success). That negative is what motivated the hull-aware, persistence-backed shield of Sec. IV-D.

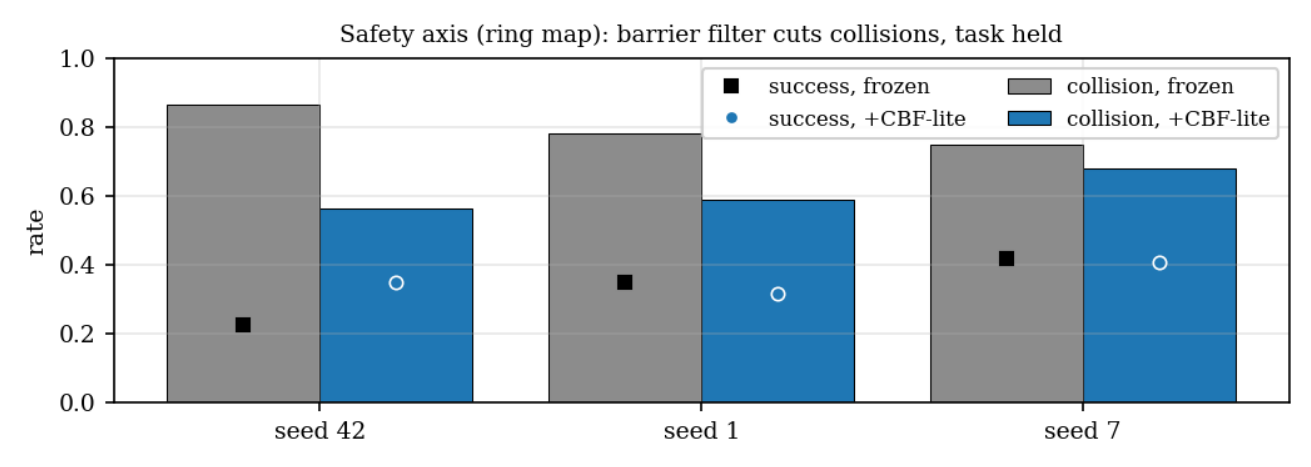


Fig. 8. CBF-lite A/B on the ring map, per seed: collision rate (bars) drops under the filter while success (markers) is held or improved. *Placement: full-width (figure*), page 7. Data: ledger §8.*

E. Safety axis, part 2: the presentation runs and the zero-contact iteration

Iteration 1 (competence arm alone). Five presentation runs reached their goals — and the exact-box audit found contact in four of them (Table II, Fig. 9). The contact decomposes into three classes: brief approach grazes (open 6 frames at ≤1.4 cm depth, mid 1, crossings 1); a sustained wedge on the hard rung (gauntlet, 917 frames, goal never reached); and post-success grinding against the goal pocket (the bulk of the full-run counts, e.g. tour 71 frames, all after the final arrival), a consequence of the frozen policy having no stop-at-goal behaviour. Reading this correctly matters: the contacts are the expected behaviour of a policy trained with contact as a recoverable event, deployed without a guarantee arm. Iteration 1 is the measurement of the framework's predicted gap, not a regression.

Iteration 2 (guarantee arm on). The same checkpoint, the same maps, the same launch procedure, shield enabled by one parameter. All five maps reach their goals with *zero* hull-penetration frames over the full recorded run (Fig. 9, Fig. 10), minimum hull clearance +0.131 to +0.287 m. Recoveries drop an order of magnitude (open 20 → 1, mid 14 → 1): the shield removes stall/blip cascades at the source instead of letting the recovery layer clean up after them. The hard rung flips from failure-with-contact to success-without-contact (29.10 s, one

recovery): removing the contact cascade lets the policy's own competent steering accumulate progress. Shield activity is sparse where the policy is competent (38–86 intervention ticks of 2400) and zero on one tour run (Fig. 12).

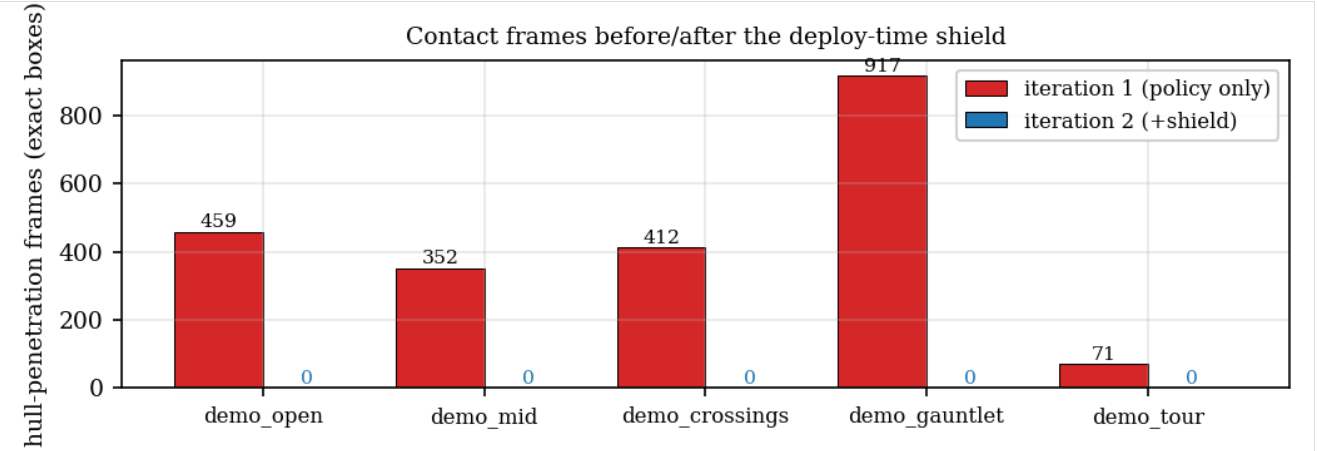


Fig. 9. Hull-penetration frames under exact wall geometry, full recorded runs, recomputed from kept traces: iteration 1 [459, 352, 412, 917, 71] versus iteration 2 [0, 0, 0, 0, 0]. Iteration-1 full-run counts include post-success grind; the approach-phase decomposition is in Table II. *Placement: full-width (figure*), page 7. Generated by make_figures.py from traces.*

Table II — Presentation runs, exact-box audit. Approach = up to first arrival at the final goal; full = entire recorded run. Iteration-2 times are policy-clock time-to-goal.

map	iter.	verdict	time	recov.	shield ticks	approach pen.	full pen.	min clr.
open	1	SUCCESS	29.3 s	20	—	6	459	−0.014 m
open	2	SUCCESS	26.2 s	1	67/2400	0	0	+0.182 m
mid	1	SUCCESS	26.2 s	14	—	1	352	−0.005 m
mid	2	SUCCESS	25.0 s	1	47/2400	0	0	+0.179 m
crossings	1	SUCCESS	33.5 s	2	—	1	412	−0.003 m
crossings	2	SUCCESS	24.6 s	1	38/2400	0	0	+0.287 m
gauntlet	1	FAIL (wedge)	—	25	—	917	917	−0.062 m
gauntlet	2	SUCCESS	29.1 s	1	86/2400	0	0	+0.158 m
tour	1	SUCCESS 3/3	46.1 s	3	—	0	71	+0.331 m
tour	2a	SUCCESS 3/3	49.9 s	0	0/2400	0	0	+0.237 m
tour	2b	SUCCESS 3/3	57.2 s	0	45/2400 (20 esc.)	0	0	+0.131 m

The livelock and its fix. The first shielded tour run of the presentation set (shield v4) cleared four maps but livelocked at tour goal 3: in the tight corner pocket, grazing hits plus dilation left no forward candidate admissible in tier 1, and the robot rotated in place 9 mm short of the success radius. Exact geometry showed the pocket was open (nearest walls 0.455/0.465 m). The two-tier barrier (Sec. IV-D) resolves it without weakening the guarantee: of two independent v5 tour runs, one threaded the pocket with zero shield interventions (49.9 s), the other rotation-locked and crept out through tier 2 (20 escape-tier ticks, 57.2 s); both are 3/3 goals with zero contact. Run-to-run ROS/sim timing selects the path; the guaranteed quantity — the contact outcome — held in both. The livelock trace is kept as an artifact.

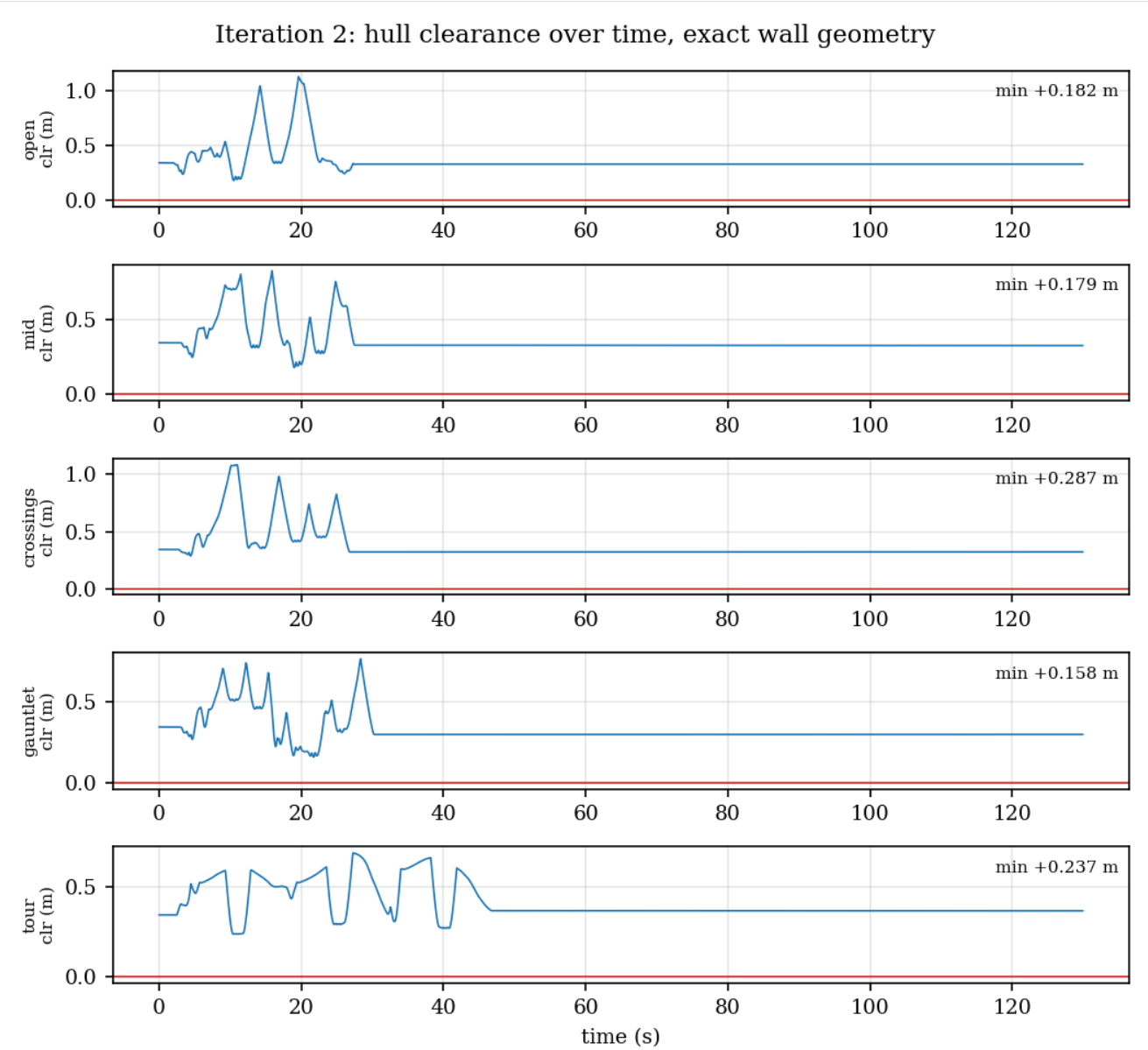


Fig. 10. Hull clearance over time for the five shielded runs, exact wall geometry. The curve never crosses zero; annotated minima match Table II. The tour panel includes two leg transitions. *Placement: full-width (figure*), page 8. Generated by make_figures.py from traces.*

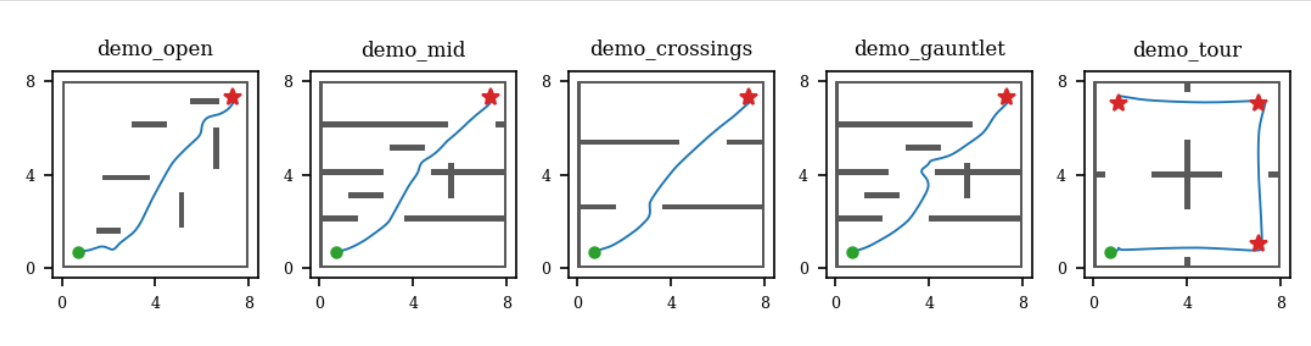


Fig. 11. Shielded trajectories on the five presentation maps (walls grey, spawn green dot, goals red stars). The gauntlet S-detour and the three-leg tour are composed by the frozen policy plus interface. *Placement: full-width (figure*), page 8. Generated by make_figures.py from traces and map JSONs.*

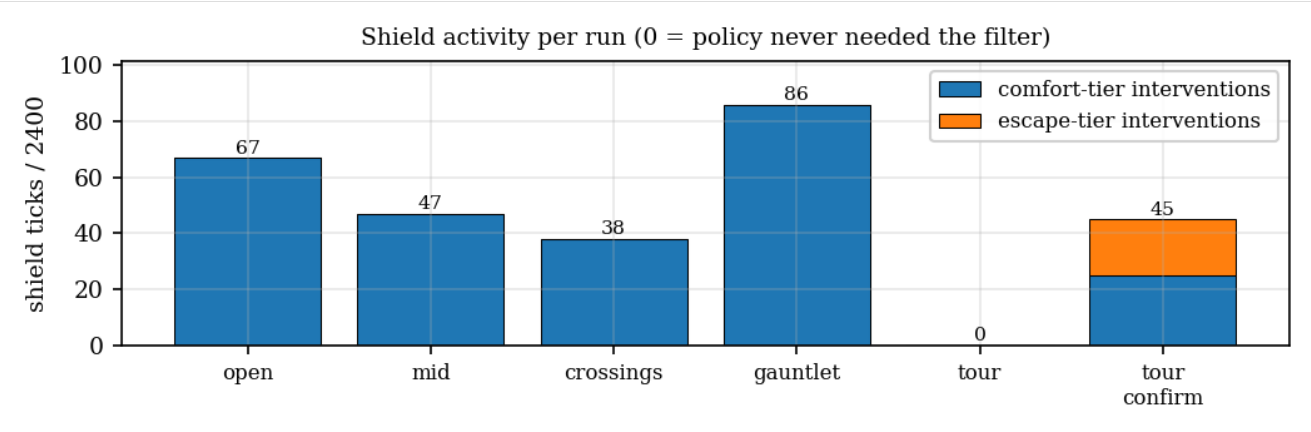


Fig. 12. Shield interventions per run, parsed from the run logs (comfort tier vs escape tier). Zero interventions on tour run 2a means the policy's own commands were provably safe at every tick; the escape tier fires only in the run that rotation-locked. *Placement: full-width (figure*), page 8. Generated by make_figures.py from logs/shield_policy_*.log.*

0:00 / 0:00 0:00 / 0:00

Video 1 (left) and Video 2 (right). Top-down renders of the shielded three-goal tour and the shielded hard rung (gauntlet), 15 Hz, LiDAR fan and waypoint overlay. *Placement: supplementary material. Full set (top-down + 3D bird's-eye for all five maps, both iterations) lives in openmover_ws/isaac/lab/videos/custommap_shield/ and .../custommap_demo/; for the printed version extract representative frames from trace_demo_tour_clip.bird3d.mp4 (3D scene, full-map framing) and trace_demo_gauntlet_clip.bird3d.mp4.*

F. Task and deployment axes

Task structure transfers through the interface: the ordered three-goal tour is solved by the frozen policy with per-leg GeoField rebuilds (46.1 s in iteration 1; 49.9/57.2 s shielded), i.e. composition happens at the interface level with zero weight changes. Deployment transfers through the message boundary: the same node stack drives a pioneer3dx plant — different wheel geometry, mass and casters — through the identical `/cmd_vel` topic, reaching an 8.07 m goal in 30.85 s. This is a deployment-level cross-embodiment result (the velocity-level interface absorbs the embodiment difference); it is not the GW latent alignment of the framework draft, which remains unexercised (Sec. VII-B). Table III consolidates every transfer axis examined in this section under a single frozen checkpoint.

Table III — Transfer ledger, frozen checkpoint throughout (condensed from ledger §3, §6–§14).

axis	mechanism	result
geometry (zero-shot)	none	rail yard 0.190 blocked succ. (−34 % vs source 0.287)
geometry + task	GeoField from shared map JSON	user maps zero-shot; ladder easy/mid 29.3/26.2 s; 3-goal tour composed
robustness	spawn jitter ±0.15 m / ±15°	4/4 success, 20.9–33.5 s
interface necessity	waypoint-blinding ablation	4/4 → 0/1, pins 5.74 m short
dynamics	DOB-lite observer	degraded 0.028 → 0.363; within 3.2 % of oracle
safety (training world)	CBF-lite cone barrier	collisions −23 %, success 0.331 → 0.358 (ring)
safety (deployment)	two-tier shield	0 contact frames on 5 maps + 2 tour runs (exact audit)
adaptation budget	retrain-from-scratch curve	zero-shot ≡ ≈238 k target demos
dynamics model	frozen Koopman lift + refit	wheel channels 0.88 → 0.57 → 0.99 (refit)
embodiment	velocity-level interface	pioneer3dx plant, 8.07 m reach, 30.85 s, zero retraining
safety at scale	shield-wrapped A*+DWA (Sec. VI-G)	56 maps × 3 arms, 708 episodes: 0.979 succ., 0.000 collision

G. The systematic campaign: 56 procedural maps, three arms

The presentation runs are hand-picked; the campaign is not. We generated 56 maps from 7 procedural families (corridor, gates, loop, fortress, competition, warehouse, highway) × 4 difficulty bins × 2 variants, and ran every map under three arms at a fixed seed with 4 spawn-yaw repeats (10 on two locked presentation

maps): the frozen trained policy under the shield, our in-house A*+DWA planner *unprotected*, and the same planner *wrapped in the shield* — the thesis arm, where a competent but uncertified controller receives guarantees from the interface alone. 236 episodes per arm, 708 total, 4.5 h wall clock, zero failed runs.

Table IV — Three-arm campaign, episode-pooled over 56 maps (236 episodes/arm; ledger §17.20). Collision = simulator contact force > 5 N; TTG = mean time-to-goal over successful episodes.

arm	success	collision	TTG (s)	shield interventions
trained + shield	0.644	0.004 (1/236)	47.5	46.6 % of ticks
A*+DWA (unprotected)	0.941	0.534	52.7	—
A*+DWA + shield (thesis arm)	**0.979**	**0.000 (0/236)**	54.8	14.5 % of ticks

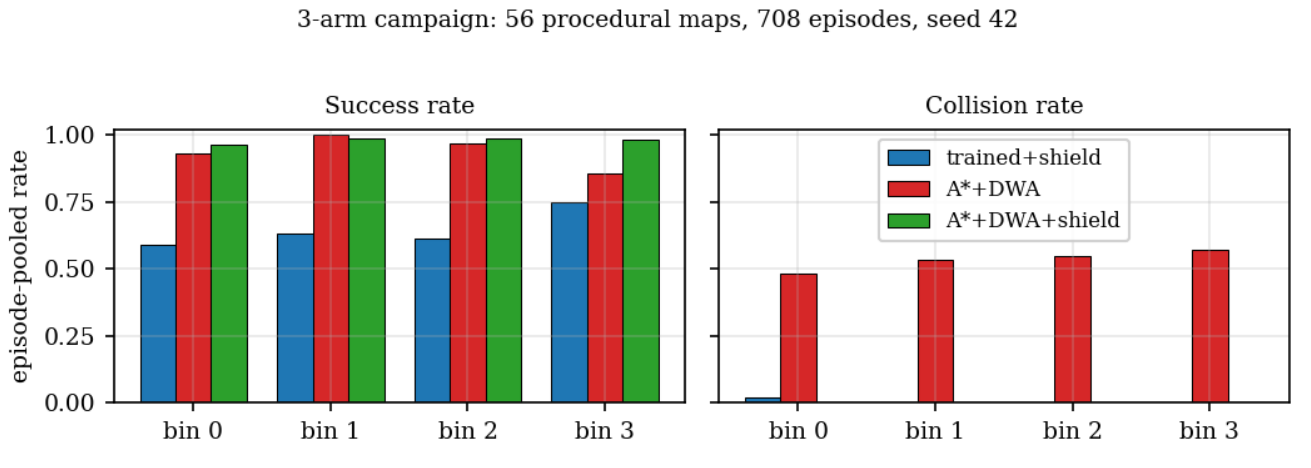


Fig. 13. Campaign per difficulty bin, episode-pooled. Unprotected collisions grow with difficulty (0.48 → 0.57) while the shielded planner is flat at 0.000 in every bin and even gains success (0.941 → 0.979 overall: stalls become escape-tier recoveries instead of timeouts). The trained arm's collisions are equally flat, but its success is family-structured (text). *Placement: full-width (figure*), page 9. Generated by make_figures.py:fig_campaign from campaign/aggregate.json.*

Three results matter. **(i) The guarantee held at scale.** The thesis arm scored zero collisions on all 56 maps — 0/236 episodes — and the off-model audit (ticks where the robot left the barrier's map-rectangle domain) counted zero over 452,404 env-steps per shielded arm. The unprotected planner, by contrast, contacted walls in 53.4 % of episodes, concentrated where geometry is tightest (fortress 0.95, warehouse 0.74, corridor 0.72) and nearly absent on the wide highway family (0.03): the failure class is corner-cutting in narrow space, exactly what the hull-aware barrier certifies against. **(ii) The shield is cheap and difficulty-flat.** It costs +2.1 s TTG (+4 %), intervenes on 14.5 % of ticks, and its collision rate does not move across difficulty bins (Fig. 13). **(iii) An honest negative on the learned arm.** The trained policy's competence is family-structured: 0.94 on competition/highway but 0.31 on gates and 0.000 on corridors (31/32 timeouts), and it accounts for the campaign's only shielded contact — the marginal 5.6 N tier-3 scrape root-caused to uncertified dither recovery (Sec. IV-D, ledger §17.19). 83 of its 84 failures are timeouts, not contacts: the shield holds safety exactly where the policy fails the task. This asymmetry is the framework's claim in one picture — guarantees come from the interface and are arm-agnostic; competence can come from anywhere, and when the learned policy's coverage runs out, a wrapped classical planner inherits the same guarantee with none of its own competence lost.

H. The held-out endgame: a map drawn by the user, unseen in training and the campaign

The campaign's 56 maps are produced by our own procedural families; a sharper test of "the guarantee comes from the interface" is a map nobody engineered. We asked a user to draw an arbitrary maze on a blank canvas — a serpentine with no straight shot from start to goal — and ran it with no retraining, no map-specific tuning and the *identical* locked protocol (10 spawn-yaw seeds, seed 42, horizon 147.5 s = path/0.25+10). The drawing rasterises to a 15.0×15.0 m world, 884 wall boxes, a 34.4 m geodesic whose narrowest corridor is ≈1.1 m (clearance ≥0.50 m along the whole path); it appears in neither the training distribution nor the campaign (ledger §17.22).

Table V — Held-out endgame: one user-drawn map, unseen in training and the campaign, same locked protocol (10 spawn-yaw seeds, seed 42; ledger §17.22). Collision = simulator contact force > 5 N; TTG = mean time-to-goal over successful episodes.

arm	success	collision	TTG (s)	shield interventions
trained + shield	0.000 (0/10, all timeout)	0.000 (0/10)	—	75.9 % of ticks
A*+DWA (unprotected)	1.000 (10/10)	0.900 (9/10)	103.3	—
A*+DWA + shield (thesis arm)	**1.000 (10/10)**	**0.000 (0/10)**	105.7	19.2 % of ticks

The three arms separate cleanly on this single unseen map (Table V), exactly along the framework's axis. **(i) The learned**

policy does not transfer to this geometry. The frozen checkpoint reaches the goal in 0/10 episodes — every seed runs out the 147.5 s horizon. This is a genuine competence failure, not truncation: both planner arms finish in ≈104 s, ≈30 % of the budget to spare, while the trained arm consumes all of it and still arrives nowhere (its tight-serpentine switchbacks are the corridor/gates gap of Sec. VI-G, now on a map no procedure produced). The shield works far harder here than in-distribution (75.9 % of ticks vs 46.6 % in Table IV) yet cannot manufacture competence; it does hold safety exactly where the policy fails — 0/10 collisions, zero out-of-bounds. **(ii) The classical planner transfers but is not safe by itself.** A*+DWA plans on the occupancy grid of *any* map, so its competence is map-agnostic: 10/10 success. Unprotected it contacts walls in 9/10 episodes, and its own DWA margin never once rejected a candidate (dwa_reject 0) — the point-model deemed every command safe while the hull and wheels clipped the ≈1.1 m corridor walls, the same hull-model gap as Sec. VII-B, now reproduced on an unseen drawing. **(iii) Wrapped in the shield, it is competent *and* guaranteed.** The thesis arm runs the identical planner (plan 1500, replan 1490, dwa_reject 0 — bit-identical to its unprotected twin) and converts 9/10 collisions into **0/10**, zero out-of-bounds, for +2.4 s (+2.3 %) TTG; the exact-box ESDF barrier intervenes on only 19.2 % of ticks to veto the corner-cuts DWA's margin waves through. This extends the campaign's thesis-arm guarantee from 0/236 to **0/246** episodes with no map-specific adjustment: on a map drawn by a person and never seen in training, the guarantee still comes from the interface, not the policy.

VII. DISCUSSION

A. What the results do and do not prove

Taken together, the results support one sentence: *a learned policy provides competence that transfers across worlds, tasks, dynamics and embodiments with frozen weights, and a mathematical interface provides the guarantees and task grounding that the learned component cannot*. Each clause is measured: transfer by the ladder, seeds and budget curve; competence-carrier by the blinding ablation; guarantees by the zero-contact audit under exact geometry; the interface/competence boundary by the shield's inability to reach any goal alone and the policy's inability to stay contact-free alone. The combination beats each part on the union of their metrics: iteration-2 runs reach what iteration-1 reached, in equal or less time, with an order of magnitude fewer recoveries and zero contact — and the 56-map campaign shows the guarantee is arm-agnostic at scale: 0/236 collisions for the shield-wrapped planner, 53.4 % for the same planner unwrapped, 1/236 for the trained policy under the same shield — and, on a held-out map drawn by a user and unseen in training (Sec. VI-H), 0/10 for the shield-wrapped planner where its unprotected twin is 9/10 and the frozen policy reaches 0/10 (0/246 cumulative).

Against the objection that classical planners already solve contact-free navigation on known maps: the campaign answers with a measurement — they do not, not by themselves. Our own competent A*+DWA planner contacts walls in over half of its 236 campaign episodes; wrapped in the shield it is contact-free on all of them and *more* successful (0.979 vs 0.941). And iteration 2's shield *is* a classical DWA-family layer — deliberately. The claim was never that a network dodges walls better than DWA. A hand-written planner does not transfer: retarget it to a new world, degraded dynamics or a new embodiment and it is rewritten and retuned, whereas the single frozen checkpoint crossed all four axes through interfaces. Conversely the shield alone reaches nothing. Neither arm alone does the job; the composition does, and the composition is what we offer.

B. Limitations and threats to validity

- **Simulation only.** No physical-robot validation; the ROS 2 rung and the second simulated embodiment are the closest proxies.
- **Observation-relative guarantee.** The shield certifies against sensed evidence on static maps. Obstacles never registered by any beam (below sensor height from the start, or occluded inside the 0.22 m near clip before first sight) are outside its evidence; margin and low speed bound but do not eliminate this. Dynamic obstacles need a bounded-staleness decay, not yet implemented.
- **Fixed-seed statistics.** The transfer table (2 k episodes per world), the 4/4 jitter repeats and the 56-map campaign (708 episodes) carry the statistics, but the campaign protocol fixes the torch seed at 42 and varies spawn yaw (4–10 repeats/map) and map geometry instead; a multi-seed campaign remains future work. The shielded videos are existence demos with frame-exact audits, not distributions. Run-to-run nondeterminism is disclosed where it changed behaviour (tour paths).
- **Protocol-relative guarantee and a hull-model gap.** The campaign's 0/236 thesis-arm collisions hold under the 10-env evaluation protocol. A single-env (`num_envs=1`) replay of the same seed on f6_b2_m00 livelocks into a wall corner and registers one >5 N contact at t≈135 s (timeout). A frame-exact hull audit shows the certified 0.44×0.28 m hull box *never* penetrates (min +0.070 m, 0/2711 frames); the contact is the drive wheels and casters, which sit ≈2 cm outside that box. The no-contact guarantee is therefore relative to (i) the evaluation protocol and (ii) the hull model — the barrier certifies a box that under-covers the true wheel envelope. Widening the certified hull to the wheel track is the direct fix (queued, not claimed); the presentation videos use the clean 10-env rerun (ledger §17.21).
- **The wedge seam remains on the hardest geometry.** Narrow-door maps (1.0 m doors, 0/3) are still beyond the frozen policy; the shield converts that failure from "collide" to "stop short", which is an acceptable failure mode but still a failure — at campaign scale the same pattern is the trained arm's corridor/gates timeouts (Sec. VI-G) and its single tier-3 uncertified scrape (ledger §17.19).
- **Mechanism-level negatives.** The quadratic Koopman lift adds little over an affine model at this scale; the single-cone CBF-lite is blind to side contacts; park-after-success is interface policy, not learned behaviour.
- **Unexercised framework arm.** GW cross-embodiment latent alignment is not tested; the embodiment result rides the velocity-level interface.

C. How this serves the "learning can be transferred" goal

Transfer learning already demonstrates that learning transfers when target data is available; our method is one answer for the regime where it is not, or where spending it is unsafe or expensive. The framework's service to the broader goal is threefold: it *prices* transfer (the adaptation-budget crossing states what frozen transfer saves in target demonstrations); it *localises* transfer (ablations and the ladder say which interface carries which axis, and where the boundary lies, in behavioural rather than visual terms); and it *certifies* transfer (the guarantee arm converts "the policy usually behaves" into "no command reaches the chassis without a proven-safe forward simulation", with the residual risk stated) — a certification that holds even on a map drawn by a user and never seen in training (Sec. VI-H), where the learned core's competence runs out but the interface's guarantee does not. None of this replaces transfer learning; it delineates the part of the cross-world burden that mathematical structure can carry while the weights stand still.

VIII. CONCLUSION AND FUTURE WORK

We presented a benchmark study of the transfer-first paradigm: one frozen differential-drive policy, trained once, redeployed across unseen geometry, task structure, degraded dynamics and a second embodiment through adaptable mathematical interfaces, and wrapped at deployment in a two-tier safety shield whose no-contact property is structural and audited frame-by-frame against exact wall geometry. The positives (zero-shot gradient, 4/4 under jitter, 0.028 → 0.363 dynamics recovery, zero contact frames, and — at campaign scale over 56 procedural maps and 708 episodes — a shield-wrapped planner at 0.979 success with 0/236 collisions where its unprotected twin collides in 53.4 %, a guarantee that carries to a held-out user-drawn map unseen in training at 10/10 success and 0/10 collisions (0/246 cumulative) where the frozen policy reaches 0/10) and the negatives (wedge seam, no-op filter variants, unhelpful lift, livelock before its fix, the trained arm's corridor/gates competence gap and its single tier-3 scrape) are reported with the same instruments. Future work, queued and not claimed: bearing-aware or barrier-shaped recovery to close the wedge seam; clearance-gated tier-3 commitment (the v7.6 lead, ledger §17.19); bounded-staleness persistence for dynamic obstacles; few-shot lift refits in closed loop; GW cross-embodiment alignment; and hardware validation, for which the stack is edge-shaped by construction (plain ROS 2 boundary, ≈14 ms CPU shield).

REFERENCES

[1] N. Jaquier *et al.*, "Transfer learning in robotics: An upcoming breakthrough? A review of promises and challenges," *Int. J. Robotics Research*, vol. 44, pp. 465–485, 2024. DOI: 10.48550/arXiv.2311.18044.

[2] A. Shakerimov, T. Alizadeh, and H. A. Varol, "Efficient sim-to-real transfer in reinforcement learning through domain randomization and domain adaptation," *IEEE Access*, vol. 11, pp. 136814–136825, 2023.

[3] T. Wang, D. Bhatt, X. Wang, and N. Atanasov, "Cross-embodiment robot manipulation skill transfer using latent space alignment," arXiv preprint arXiv:2406.01968, 2024.

[4] D. Kalaria and K. Karydis, "Disturbance observer-based control barrier functions with residual model learning for safe reinforcement learning," arXiv preprint arXiv:2410.06570, 2024.

[5] Y.-H. Chen, S. Liu, W. Xiao, C. Belta, and M. Otte, "Control barrier functions via Minkowski operations for safe navigation among polytopic sets," in *Proc. IEEE 64th Conf. Decision and Control (CDC)*, 2025. DOI: 10.1109/CDC57313.2025.11312188.

[6] L. Shi, Z. Liu, and K. Karydis, "Koopman operators for modeling and control of soft robotics," *Current Robotics Reports*, vol. 4, pp. 23–31, 2023. DOI: 10.1007/s43154-023-00099-8.

[7] A. K. Li, T. C. Silva, V. Edwards, V. Kumar, and M. A. Hsieh, "KoopMotion: Learning almost divergence free Koopman flow fields for motion planning," *Proc. Machine Learning Research (PMLR)*, vol. 305, 2025. DOI: 10.48550/arXiv.2509.09074.

[8] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

[9] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proc. 18th European Control Conf. (ECC)*, 2019, pp. 3420–3431.

[10] S. Ross, G. J. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. 14th Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

[11] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, and M. Hutter, "Orbit: A unified and modular framework for robot learning environments," in *Proc. 5th Conf. Robot Learning (CoRL)*, 2021.

[12] M. Mittal *et al.* (NVIDIA), "Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning," technical report, arXiv:2511.04831, 2025.

[13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv preprint arXiv:1707.06347, 2017.

[14] S. L. Brunton, M. Budišić, E. Kaiser, and J. N. Kutz, "Modern Koopman theory for dynamical systems," *SIAM Review*, vol. 64, no. 4, pp. 729–840, 2022.

[15] W.-H. Chen, D. J. Ballance, P. J. Gawthrop, and J. J. Gribble, "A nonlinear disturbance observer for robotic manipulators," *IEEE Trans. Industrial Electronics*, vol. 47, no. 4, pp. 806–814, 2000.

[16] A. V. Pitkevich and I. Makarov, "A survey of sim-to-real transfer methods for robotic manipulation," in *Proc. IEEE 22nd Int. Symp. Intelligent Systems and Informatics (SISY)*, 2024. DOI: 10.1109/SISY62279.2024.10737545.

[17] D. Y. Patrashko and V. Gurau, "Machine learning-powered vision for robotic inspection in manufacturing: A review," *Sensors*, vol. 26, no. 3, p. 788, 2026. DOI: 10.3390/s26030788.

[18] Q. Li, F. Cicirelli, A. Vinci, A. Guerrieri, W. Qi, and G. Fortino, "Quadruped robots: Bridging mechanical design, control, and applications," *Robotics*, vol. 14, no. 5, p. 57, 2025. DOI: 10.3390/robotics14050057.

[19] G. Pan *et al.*, "RoboDuet: A framework affording mobile-manipulation and cross-embodiment," arXiv preprint arXiv:2403.17367, 2024.

[20] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm7407, 2022.